Parallel Ant Colony Optimization on Fractal Landscapes

NAN LIN, ENSTA Paris, France

1 Introduction

Swarm intelligence algorithms solve complex search and optimization problems by coordinating many simple agents through local interactions and environment-mediated communication. Among them, **Ant Colony Optimization (ACO)** is a canonical paradigm: agents (ants) explore the domain while depositing and sensing pheromone, and the pheromone dynamics in turn biases future decisions. This feedback loop creates a computation pattern that is highly regular and massively repeated, making ACO not only an optimization method but also a compelling simulation workload.

In this work, we use ant-based foraging on a fractal (plasma-generated) landscape as a representative ACO-style workload, where movement cost varies spatially and obstacles may exist on a 2D grid. Crucially, our goal is to characterize and exploit the parallelization opportunities inherent in the algorithm and its simulator. Each iteration consists of (i) simulating the motion and local decisions of many ants, (ii) updating pheromone deposition, and (iii) applying pheromone evaporation and diffusion (if enabled) over the grid. These steps expose two dominant forms of parallelism: agent-level parallelism (across ants) and cell-level parallelism (across grid locations). At the same time, they induce well-defined synchronization and reduction points (e.g., concurrent pheromone writes and global evaporation), which become the main performance bottlenecks.

Therefore, we study how to restructure the simulator to better match modern hardware: vectorization/SIMD on CPUs, shared-memory parallelism via multithreading, and—when relevant—distributed-memory decomposition of the grid and pheromone field. Our analysis focuses on computational intensity, memory bandwidth pressure, contention on pheromone updates, and the cost of global synchronization, providing guidelines for implementing scalable parallel ACO simulators on heterogeneous platforms.

Contributions. The main contributions of this report are summarized as follows:

- **Workflow characterization of an ACO-style simulator.** We instrument the simulator to measure the execution time of its main components and show that the ant movement phase dominates the runtime, while pheromone evaporation represents the main secondary cost.
- **SIMD-based vectorization of key computational kernels.** We reorganize the ant state representation into a vector-friendly layout and introduce SIMD-enabled kernels for regular grid operations such as terrain normalization and pheromone evaporation, leading to measurable performance improvements.
- **Shared-memory parallelization with OpenMP.** We exploit agent-level parallelism by distributing ant updates across CPU threads while preserving correctness through thread-local accumulation and merging of pheromone marks, significantly reducing the cost of the dominant computation phase.
- **Evaluation of a replicated-environment MPI strategy (Premiere façon).** We implement and experimentally evaluate the first distributed-memory approach where each process simulates a subset of ants while maintaining a full copy of the environment. Our measurements highlight the limitations of this strategy due to the high cost of global pheromone synchronization.

Author's Contact Information: Nan LIN, ENSTA Paris, Palaiseau, France, nan.lin@ensta.fr.

- **A shared-memory analogue of the sub-map ownership strategy (Seconde façon).** We design and implement a locality-aware parallel organization in which the map is partitioned into row-wise sub-domains, ants are dynamically assigned to their corresponding owners, and only cross-owner pheromone marks are exchanged. Experiments show that this ownership-based organization improves scalability in multi-threaded executions by reducing contention and unnecessary global sharing.

2 Background

2.1 Swarm Intelligence and Ant Colony Optimization

Swarm intelligence refers to algorithms in which a population of simple agents collectively solves a task through local rules and indirect coordination. ACO is a prominent instance, where agents deposit a *pheromone* signal to bias future decisions. Classical ACO typically operates on discrete graphs: ants stochastically construct candidate solutions (paths) based on pheromone intensity and heuristic desirability, and pheromone updates reinforce good solutions while evaporation prevents premature convergence.

Our setting differs from graph-based ACO in two aspects. First, the environment is a 2D grid whose traversal cost varies per cell, so the notion of a "short" path depends on accumulated traversal time rather than hop count. Second, we adopt a reactive update rule in which pheromone values propagate locally based on neighborhood statistics, enabling agents to navigate without storing explicit routes.

2.2 Cost-Weighted Grid Environment

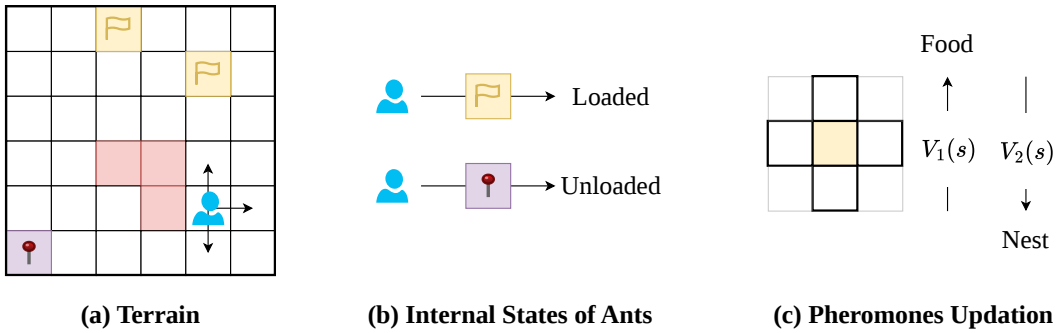

Fig. 1. Illustration of the cost-weighted grid environment

We model the world as a Cartesian grid. As is shown in Fig. 1, each cell s is associated with: (i) a traversal cost $c(s) \in [0, 1]$ representing the time required to enter or cross the cell, (ii) a cell type indicating whether it is free space, an obstacle (non-traversable), the nest, or a food source. An ant is located at a grid cell and can move to one of its four von Neumann neighbors if that neighbor is traversable.

2.3 Dual-Pheromone Reactive Foraging Model

The simulator maintains two pheromone fields over the grid: $V_1(s)$ encourages exploration toward food, and $V_2(s)$ encourages returning to the nest once food is found. Each ant has a binary internal state: *unloaded* (not carrying food)

or *loaded* (carrying food). When an unloaded ant reaches a food cell, it becomes loaded; when a loaded ant reaches the nest, it becomes unloaded and a global food counter increases.

At each time step, pheromones are updated locally using only neighborhood aggregate information. Let $N(s)$ denote the four neighbors of s. For pheromone type $i \in \{1, 2\}$, define:

$$\max_i(N(s)) \triangleq \max_{s' \in N(s)} V_i(s'), \qquad avg_i(N(s)) \triangleq \frac{1}{4} \sum_{s' \in N(s)} V_i(s'). \tag{1}$$

Then the update rule is:

$$V_1(s) \leftarrow \begin{cases} 1, & \text{if } s \text{ is a food cell,} \\ \alpha \ \max_1(N(s)) + (1-\alpha)\, avg_1(N(s)), & \text{otherwise,} \end{cases} \tag{2}$$

$$V_2(s) \leftarrow \begin{cases} 1, & \text{if } s \text{ is the nest cell,} \\ \alpha \ \max_2(N(s)) + (1-\alpha)\, avg_2(N(s)), & \text{otherwise,} \end{cases} \tag{3}$$

where $\alpha \in [0, 1]$ controls the amount of "noise" (or smoothing) in pheromone propagation.

Ant movement combines exploration and exploitation. With probability $\varepsilon \in [0, 1]$ (exploration rate), the ant moves to a uniformly random traversable neighbor. With probability $1 - \varepsilon$, the ant follows the steepest local pheromone direction:

$$s_{t+1} \in \begin{cases} \arg\max_{s' \in N(s_t)} V_1(s'), & \text{if unloaded,} \\ \arg\max_{s' \in N(s_t)} V_2(s'), & \text{if loaded.} \end{cases} \tag{4}$$

To account for heterogeneous terrain, each ant has a movement budget of 1 per time step, and entering a cell consumes $c(s)$ from that budget; an ant may perform multiple sub-moves within one time step until its budget is exhausted.

Finally, pheromones evaporate globally to avoid saturation:

$$V_i(s) \leftarrow \beta\, V_i(s), \quad i \in \{1, 2\}, \tag{5}$$

where $\beta \in (0, 1]$ is typically close to 1.

2.4 Fractal Terrain via Plasma Generation

To generate a spatially correlated cost field $c(s)$, we employ a stochastic plasma generation procedure inspired by midpoint displacement. The grid is partitioned into overlapping sub-grids of size $ns = 2^k + 1$, arranged in an $n \times n$ tiling. Corner values are initialized using a seed-dependent pseudo-random generator, ensuring that incremental deviations are bounded by a parameter d times the current scale. Each sub-grid is recursively subdivided; midpoints of edges and the center are assigned by averaging parent vertices plus a bounded random deviation that decreases with scale (e.g., proportional to $d \cdot ns/2$ at the next recursion level). After recursion, values are normalized to $[0, 1]$, producing a fractal-like terrain where local gradients are controlled by d.

2.5 Parallelization Opportunities

The simulator exposes natural parallel structure: (i) ant updates and movements are parallel across ants except for concurrent writes to pheromone fields, (ii) evaporation and neighborhood-based pheromone propagation are parallel across grid cells. These properties motivate both shared-memory parallelization (e.g., OpenMP over ants and/or cells) and distributed-memory decompositions. We consider two distributed strategies: *replicated environment* (each process

stores the full grid and simulates a subset of ants, followed by a reduction/merge of pheromone fields), and *domain decomposition* (each process owns a sub-domain of the grid and exchanges halo/border data while migrating ants crossing boundaries). These strategies trade memory footprint, load balance, and communication volume, and will be evaluated experimentally.

3 Experiment Setup

3.1 Hardware Platform

All experiments were conducted on a MacBook Air (15-inch, 2025) laptop equipped with an Apple M4 system-on-chip (SoC). The processor integrates a 10-core CPU (4 performance cores and 6 efficiency cores), a 10-core integrated GPU, and a 16-core Neural Engine. The system is configured with 16 GB of unified memory shared by the CPU and GPU, providing a peak memory bandwidth of approximately 120 GB/s. The operating system is macOS running on Apple Silicon.

The detailed hardware characteristics are summarized in Table 1.

Table 1. Experimental hardware platform

Component	Specification
CPU	Apple M4 (10 cores: 4 performance + 6 efficiency)
CPU Frequency	up to ~4.4 GHz
GPU	Integrated Apple GPU (10 cores)
GPU Compute Throughput	~4.4 TFLOPS (FP32 peak)
Neural Engine	16-core Apple Neural Engine
Memory	16 GB unified memory
Memory Bandwidth	120 GB/s
Architecture	Apple Silicon (3 nm SoC)
Operating System	macOS (Apple Silicon)

3.2 Software Environment

The simulator is implemented in `C++17`. The project is organized as a native C++ codebase and compiled using `g++` through a `Makefile`-based build system.

Graphical visualization of the simulation is implemented using the `SDL2` library, which provides window management and rendering capabilities for displaying the grid environment and ant movements in real time.

The compilation configuration enables support for shared-memory parallelism through `OpenMP` (compiled with the `-fopenmp` flag).

On Linux platforms, the project also links against the POSIX threading library (`pthread`), allowing thread-based parallel execution when required.

The codebase is compiled with optimization flags enabled (e.g., `-O3`) to ensure that the experimental results reflect realistic performance characteristics of the implementation.

3.3 Simulation Workflow

The project implements a self-contained simulation workflow that generates the environment, runs the ant colony simulation iteratively, and visualizes the results in real time. Unlike typical data-processing pipelines, the system does

not rely on external datasets; instead, both the environment and the agent dynamics are generated internally during execution.

First, the project is compiled using a `Makefile`-based build system located in the `src` directory. The build process produces the executable `ant_simu.exe`, which links against the `SDL2` library for graphical rendering.

At program startup, the main function initializes the simulation parameters and system state. This includes initializing the SDL runtime, setting the random seed, configuring algorithmic parameters such as the number of ants, the exploration probability ε, and the pheromone parameters α and β. The coordinates of the nest and food sources are also defined during this stage.

The simulation environment is then generated procedurally. A fractal terrain is created using the recursive plasma generation algorithm implemented in `fractal_land.cpp`. The resulting terrain represents the traversal cost of each grid cell. After generation, the cost values are normalized using a min–max normalization step to ensure that traversal costs lie within the interval $[0, 1]$.

Next, the agent population and pheromone fields are initialized. An array of ants is created, with each ant initially placed at a random location in the grid. Two pheromone maps are allocated to represent the two navigation signals used in the model: one guiding ants toward food and the other guiding them back to the nest. The nest and food cells act as persistent pheromone sources, while boundary cells are marked as non-traversable.

The main simulation then proceeds through an iterative loop. At each iteration, the system first processes SDL events to handle user interaction and window events. The core update is performed by the `advance_time(...)` routine, where each ant executes its movement rule. With probability ε, an ant explores by selecting a random neighboring cell; otherwise, it follows the direction of the strongest pheromone gradient. Movement speed is influenced by the terrain traversal cost. When an ant reaches a food source it becomes loaded, and when it returns to the nest the food is delivered and the global food counter is incremented.

After all ants have updated their states, the pheromone fields are updated globally. This includes pheromone evaporation and diffusion steps implemented through the `do_evaporation()` and `update()` functions, which maintain the dynamic pheromone landscape used for navigation.

Finally, the current simulation state is rendered in real time using the SDL-based renderer. The visualization displays the terrain and ant positions, the intensity of the two pheromone fields, and a plot of the cumulative number of food deliveries over time. When the first successful food delivery occurs, the program prints the iteration number to the terminal.

The simulation continues until the user closes the graphical window, at which point the SDL environment is shut down and the program terminates.

4 Experiments

4.1 Workflow Benchmark

To understand the computational cost of the simulator, we instrumented the code with fine-grained timers that measure the execution time of each stage of the workflow. Each experiment was executed for 2000 iterations and repeated ten times. Table 2 reports the average execution time of each stage and is illustrated in Fig. 2.

The benchmark results highlight the dominant computational components of the simulator. The initialization stage is relatively inexpensive, accounting for less than 1% of the total runtime. Within this stage, the most expensive step is the generation of the fractal terrain, which requires approximately 18.8 ms on average.

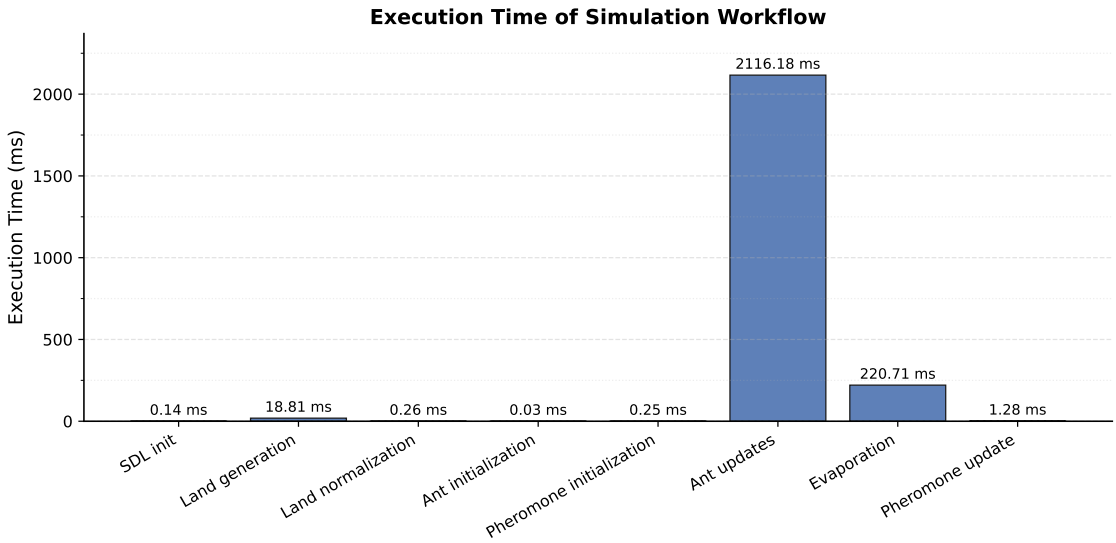

Fig. 2. Average execution time of the main workflow stages over 10 runs (–iterations 2000, –ants 5000, rendering disabled).

Table 2. Average execution time of the main workflow stages

Stage	Time (ms)	Share (%)
Initialization		
SDL initialization	0.138	0.01
Fractal terrain generation	18.808	0.80
Terrain normalization	0.264	0.01
Ant initialization	0.032	< 0.01
Pheromone initialization	0.246	0.01
Simulation loop (2000 iterations)		
Ant movement update	2116.18	89.7
Pheromone evaporation	220.71	9.4
Pheromone propagation update	1.28	0.05
Total simulation time	2338.31	99.2
Total runtime	2357.94	100

The simulation loop clearly dominates the total runtime. Across 2000 iterations, the simulator spends on average 2338 ms in the main update loop. Among the components of this loop, the ant movement update is by far the most expensive operation, accounting for approximately 89.7% of the total runtime. This cost corresponds to the repeated evaluation of movement decisions and terrain traversal for each ant.

Pheromone evaporation represents the second largest cost, contributing roughly 9.4% of the runtime. This step involves scanning the entire grid and applying the evaporation factor to each cell.

In contrast, the pheromone propagation update has negligible cost (less than 0.1% of the runtime). This indicates that the simulator is primarily limited by the per-agent computations rather than by global grid updates.

These measurements confirm that the dominant parallelization opportunity lies in the ant update phase. Since each ant evolves independently during a time step, this component exposes substantial agent-level parallelism, making it a natural target for shared-memory parallelization using techniques such as OpenMP.

Table 3. Scalability results with increasing number of ants (grid 513 × 513, 2000 iterations).

Ants	Ant updates (ms)	Evaporation (ms)	Update (ms)	Total (ms)	Food delivered
1000	788.8	284.1	3.88	1096.8	0
2500	1900.6	274.1	1.26	2194.5	0
5000	3713.6	272.6	1.26	4006.0	0
10000	7495.4	284.9	1.37	7802.7	30
20000	14932.6	299.2	1.51	15252.5	2270

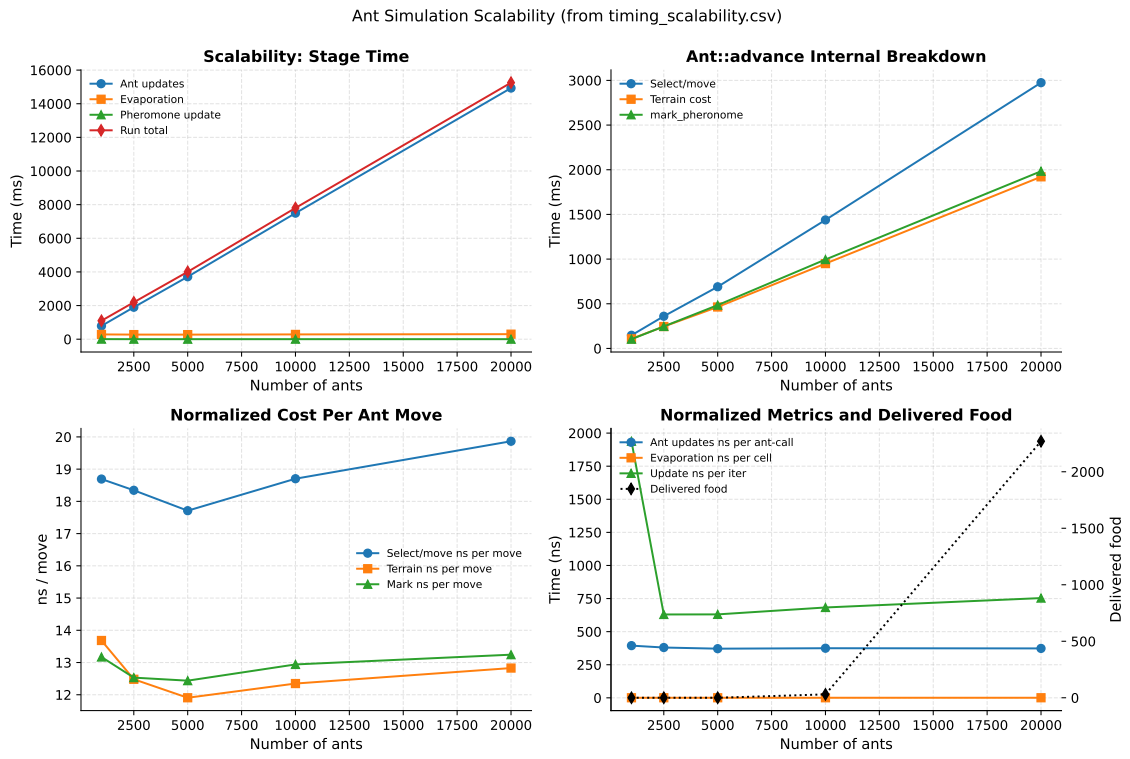

Fig. 3. Average execution time of the main workflow stages

Analysis of the dominant computation and scalability. The scalability measurements confirm that the dominant cost of the simulator originates from the ant movement phase. As shown in Table 3, the total runtime increases almost linearly with the number of ants: the execution time grows from roughly 1.1 s for 1000 ants to more than 15 s for 20000 ants. This near-linear trend indicates that the workload is primarily proportional to the number of agents rather than to a fixed global cost.

A closer inspection of the timing breakdown reveals that almost the entire increase comes from the ant update stage. The time spent updating ants grows from 788 ms to 14933 ms when increasing the population from 1000 to 20000 agents, whereas the pheromone evaporation phase remains nearly constant at around 270–300 ms. This behaviour is expected because evaporation scans the grid once per iteration and therefore depends only on the grid size, which remains fixed in the experiment. Similarly, the pheromone propagation update represents a negligible cost (about 1–1.5 ms) because it mostly consists of simple buffer exchanges and boundary resets.

The measurements also show that the cost of processing a single ant remains stable across all configurations. The average execution time per ant update is approximately 0.37–0.39 μs, indicating that the algorithm does not exhibit noticeable degradation as the population grows. Consequently, the overall runtime increase can be explained almost entirely by the linear growth in the number of agent updates.

As is shown in Fig. 3, inside the `ant::advance` function, several sub-operations contribute to the cost of a movement step. Fine-grained measurements indicate that the most expensive operation is the direction selection stage (about 18–20 ns per move), which involves reading neighboring pheromone values, evaluating the exploration–exploitation rule, and generating a random choice. The terrain traversal cost update and pheromone marking operations are slightly cheaper (around 12–13 ns each). This breakdown suggests that the decision logic itself is the main computational hotspot within each movement step.

These observations highlight two important characteristics of the current workflow. First, the majority of the computation scales with the number of ants and therefore grows linearly with the simulation size. Second, a smaller portion of the runtime corresponds to fixed grid-wide operations such as pheromone evaporation, which remain constant for a given environment size. As the number of agents increases, the relative contribution of the ant update stage therefore becomes increasingly dominant.

The current implementation also reveals several sources of serialization. The main simulation loop executes each iteration sequentially, performing ant updates, pheromone evaporation, and pheromone propagation in strict order. Within the ant phase, all agents are processed sequentially in a single loop, even though their movement decisions are independent. Similarly, the grid traversals used during evaporation and propagation are also implemented as sequential nested loops.

Despite this sequential structure, the algorithm exposes substantial inherent parallelism. In particular, ant updates during a given iteration are independent and can be executed concurrently. The loop over the agent population is therefore a natural candidate for data-parallel execution. However, introducing parallelism requires careful handling of shared data structures. Multiple ants may attempt to deposit pheromone on the same grid cell during the same iteration, which would introduce data races in a parallel implementation. Similarly, the global counter used to track delivered food would require atomic operations or thread-local aggregation.

Overall, the scalability measurements combined with the structural analysis indicate that the ant movement phase is both the dominant computational cost and the component whose complexity grows with the simulation size. Future optimizations should therefore focus primarily on parallelizing the agent update loop and improving the efficiency of the movement decision logic while introducing appropriate synchronization mechanisms for pheromone updates and shared counters.

4.2 Effect of SIMD Vectorization

To evaluate the impact of instruction-level vectorization, we compare the baseline scalar implementation (`parallel=0, vectorized=0, simd=0`) against the vectorized configuration (`parallel=0, vectorized=1, simd=1`) under the same

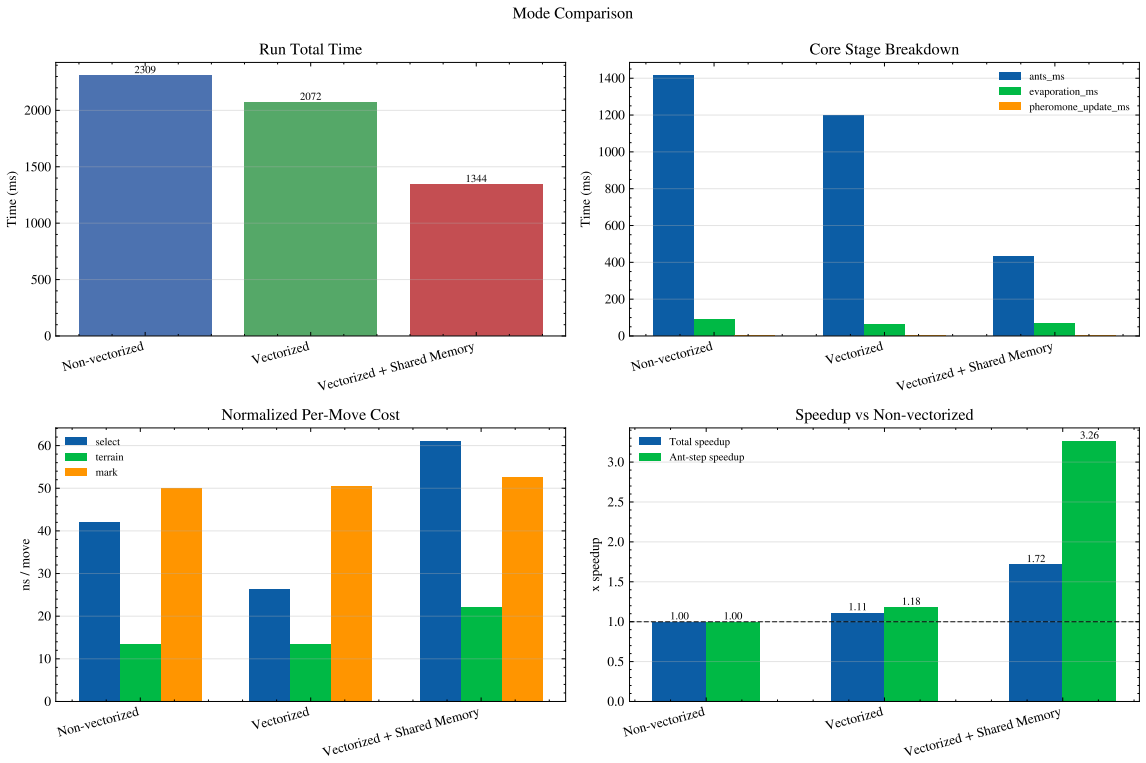

Fig. 4. **Performance comparison across execution modes (mode_comparison_scienceplots).** Experimental setup: 5000 ants on a 513 × 513 grid for 800 iterations, with 5 benchmark runs per mode and rendering disabled (`render=0`). Reported values correspond to the **median** across runs from each CSV file. The compared modes are *Non-vectorized* (scalar, single-thread), *Vectorized* (SIMD, single-thread), and *Vectorized + Shared Memory* (SIMD combined with OpenMP shared-memory parallelism using 10 threads). (a) Total runtime (`run_total_ms`), representing the end-to-end execution cost. (b) Core-stage runtime breakdown (`ants_ms`, `evaporation_ms`, `pheromone_update_ms`), highlighting where execution time is spent in the simulation workflow. (c) Normalized per-move costs (`ant_select_ns_per_move`, `ant_terrain_ns_per_move`, `ant_mark_ns_per_move`), reflecting per-step computational efficiency. (d) Speedup relative to the Non-vectorized baseline (both total runtime and ant-update stage), quantifying the performance gains from SIMD vectorization and thread-level parallelism.

workload: 5000 ants, grid size 513 × 513, and 800 iterations. The vectorized version combines two complementary changes. First, the ant state representation is reorganized into a more vector-friendly layout. Second, explicit SIMD directives are introduced in deterministic data-parallel kernels, namely terrain normalization, pheromone evaporation, and boundary-condition updates.

As shown in Table 4, SIMD vectorization produces a clear but moderate end-to-end improvement. The total runtime decreases from 2305.01 ms to 2095.50 ms, corresponding to a speedup of about 1.10×. The main loop time (`advance_time_ms`) shows a similar improvement, decreasing by about 9.11%. This result is important: even though only selected kernels are explicitly vectorized, the overall simulation already benefits measurably.

The strongest SIMD benefit appears in the grid-wide evaporation kernel. Its runtime drops from 88.75 ms to 63.03 ms, i.e., a speedup of about 1.41×, and the normalized cost per processed cell decreases from 0.4215 ns to 0.2994 ns. This is exactly the kind of loop that SIMD likes: the computation is regular, contiguous, branch-light, and free of cross-iteration

Table 4. Performance comparison between scalar and SIMD-vectorized execution (5000 ants, 513 × 513, 800 iterations).

Metric	Scalar	Vectorized+SIMD	Speedup	Reduction (%)
Advance time (ms)	2284.50	2076.36	1.10×	9.11
Ant update time (ms)	1407.43	1209.04	1.16×	14.10
Evaporation time (ms)	88.75	63.03	1.41×	28.98
Pheromone update (ms)	0.53	0.54	0.99×	-1.33
Run total (ms)	2305.01	2095.50	1.10×	9.09
Ant time per call (μs)	0.3519	0.3023	1.16×	14.10
Select move per move (ns)	41.91	26.71	1.57×	36.27
Terrain cost per move (ns)	13.22	13.64	0.97×	-3.15
Mark pheromone per move (ns)	50.14	51.16	0.98×	-2.03
Evaporation per cell (ns)	0.4215	0.2994	1.41×	28.98

dependencies. In other words, evaporation is structurally a vector machine workload, so the explicit `#pragma omp simd` annotation exposes parallelism the compiler can exploit effectively.

The ant update phase also improves, but more modestly. Its runtime decreases from 1407.43 ms to 1209.04 ms, corresponding to a speedup of about 1.16×. At a finer granularity, the movement-selection component is the part that benefits the most: its average cost falls from 41.91 ns to 26.71 ns per move. This suggests that the vectorized data layout improves not only arithmetic throughput, but also memory access regularity during direction selection. By contrast, terrain-cost evaluation and pheromone-marking remain nearly unchanged, which is consistent with their more irregular access pattern and stronger dependence on control flow and writes.

A useful way to interpret this result is the following. SIMD does not "magically accelerate the whole algorithm"; it mainly helps where the computation already looks like a dense array transform. Therefore, the grid-based kernels benefit strongly, while the ant-level logic benefits only partially. This asymmetry explains why the end-to-end gain remains around 10% even though the evaporation kernel itself improves by nearly 29%: the overall runtime is still dominated by ant motion, whose control-heavy behavior limits vector efficiency.

4.3 Effect of OpenMP Shared-Memory Parallelization

We next evaluate the shared-memory implementation based on OpenMP. In this mode, the simulation uses the vectorized ant representation and SIMD-enabled kernels, while ant traversal is further parallelized across CPU threads using `#pragma omp parallel` and `#pragma omp for schedule(static)`. To preserve correctness, each thread accumulates pheromone marks and food counters in private buffers, which are merged after the parallel region. This design avoids write races while retaining the original simulation semantics.

Table 5 shows that shared-memory parallelization is the dominant optimization. Compared with the scalar baseline, the OpenMP version reduces total runtime from 2305.01 ms to 1354.76 ms, corresponding to an overall speedup of about 1.70×. Even relative to the already vectorized version, OpenMP still provides an additional speedup of about 1.55×. The same pattern appears in the main loop timing, where `advance_time_ms` decreases from 2284.50 ms to 1335.19 ms.

The most striking effect is on the ant update phase. Its execution time falls from 1407.43 ms in the scalar version to 437.22 ms in the vectorized+OpenMP version, which corresponds to a speedup of about 3.22×. Since the ant phase is the dominant cost in the original program, this reduction is exactly why the total runtime drops so substantially. The

Table 5. Performance comparison across scalar, vectorized, and vectorized+OpenMP modes (5000 ants, 513 × 513, 800 iterations).

Metric	Scalar	Vectorized+SIMD	Vec+SIMD+OpenMP	Speedup vs. Scalar	Speedup vs. Vec
Threads	1	1	10	–	–
Advance time (ms)	2284.50	2076.36	1335.19	1.71×	1.56×
Ant update time (ms)	1407.43	1209.04	437.22	3.22×	2.77×
Evaporation time (ms)	88.75	63.03	67.39	1.32×	0.94×
Pheromone update (ms)	0.53	0.54	0.58	0.90×	0.92×
Run total (ms)	2305.01	2095.50	1354.76	1.70×	1.55×
Ant time per call (μs)	0.3519	0.3023	0.1093	3.22×	2.77×
Select move per move (ns)	41.91	26.71	61.67	0.68×	0.43×
Terrain cost per move (ns)	13.22	13.64	22.04	0.60×	0.62×
Mark pheromone per move (ns)	50.14	51.16	52.83	0.95×	0.97×

result confirms the structural analysis from the previous subsection: the most profitable source of parallelism is not inside a single ant, but across ants.

Interestingly, the normalized per-move sub-timers inside the ant phase become larger in the OpenMP experiment (e.g., movement selection rises to about 61.67 ns per move). This does *not* mean that the implementation became slower in absolute terms. Rather, it indicates that once the outer loop is parallelized, these fine-grained timers start to reflect parallel overheads such as thread scheduling, private-buffer management, merge cost, cache effects, and measurement distortion under concurrent execution. The true performance indicator here is the wall-clock time of the whole ant phase, and that quantity improves strongly.

The evaporation kernel behaves differently. Compared with the scalar baseline, it is still faster in the OpenMP run (67.39 ms vs. 88.75 ms), but it is slightly slower than the single-threaded vectorized version (67.39 ms vs. 63.03 ms). This is a useful reminder that not every regular loop benefits further from thread-level parallelization on a small machine. For bandwidth-bound kernels with low arithmetic intensity, SIMD may already capture most of the available speedup, while additional threading can introduce overheads or memory-system contention. On this workload, OpenMP is therefore most valuable for the ant traversal phase rather than for the already-efficient grid-wide SIMD kernels.

Overall, these results reveal a layered optimization picture. SIMD improves the regular inner loops, especially evaporation, and also slightly reduces the cost of the ant phase through better data layout and lower per-move decision cost. OpenMP then attacks the much larger source of work by distributing ants across cores while preserving correctness via thread-local accumulation. The combination of both techniques yields the best result, but their contributions are not symmetric: SIMD improves efficiency inside regular kernels, whereas shared-memory parallelization removes the main source of sequential execution in the simulator.

4.4 Effect of MPI Distributed-Memory Parallelization (Première Façon)

We finally evaluate the first distributed-memory strategy proposed in the project statement, namely the *replicated-environment* MPI design. In this implementation, each MPI process stores the full environment, including the terrain and the two pheromone fields, while only a subset of ants is assigned to each process. The ant population is partitioned evenly by rank, so the computational work associated with ant traversal is, in principle, well balanced.

However, this strategy introduces a crucial synchronization requirement. Because ants owned by different processes may deposit pheromone on the same cell during the same iteration, local pheromone states diverge after the ant-update

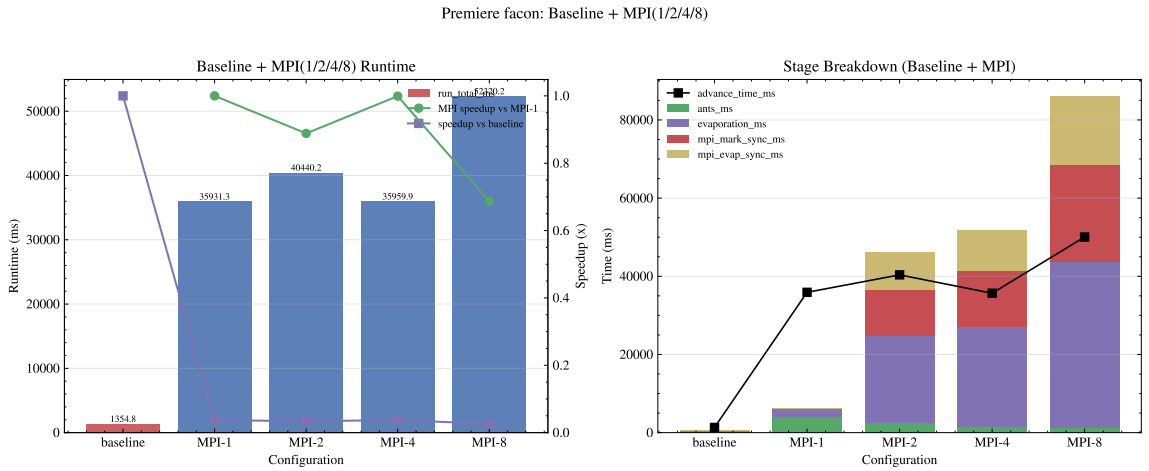

Fig. 5. **MPI vs. shared-memory baseline runtime and relative performance (Première façon).** The left panel shows the end-to-end runtime (`run_total_ms`) of the MPI replicated-environment implementation for 1, 2, 4, and 8 processes; the dashed horizontal line marks the shared-memory baseline obtained from the vectorized+OpenMP configuration. The right panel reports both the MPI speedup relative to the 1-process MPI case, $T_{\mathrm{MPI}}(1)/T_{\mathrm{MPI}}(p)$, and the relative performance ratio between the shared-memory baseline and MPI, $T_{\mathrm{shared}}/T_{\mathrm{MPI}}(p)$. Experiments use the same workload for MPI scaling (`nb_ants=5000`, `iterations=800`), with one core per MPI process (`OMP_NUM_THREADS=1`, `-threads 1`); values are averages extracted from `timing_mpi_speedup_fair.csv`, while the shared-memory baseline is taken from the avg row of `timing_vectorized_openmp.csv`. The results show that this first MPI strategy scales poorly on the tested platform, as communication and synchronization overhead dominate at larger process counts.

phase. To restore a consistent global view, the implementation performs a global `MPI_Allreduce` with the `MAX` operator over the full pheromone buffer after marking. In addition, the evaporation phase is itself partitioned by rows across processes, but the partial evaporated buffers must again be merged through another full-buffer `MPI_Allreduce(MAX)`. As a result, each iteration contains two global synchronizations over the full pheromone map.

Table 6. MPI scalability results for the first distributed-memory strategy (replicated environment, one core per process, 5000 ants, 513 × 513, 800 iterations).

Processes	Run total (ms)	Speedup	Advance time (ms)	Ants (ms)	Evaporation (ms)	MPI sync total (ms)
1	35931.3	1.000	35897.3	4046.28	2201.17	0.00
2	40440.2	0.889	40364.3	2470.86	22465.8	21169.6
4	35959.9	0.999	35671.7	1460.42	25598.5	24849.8
8	52320.2	0.687	50044.4	1138.52	42692.8	42217.0

For clarity, the last column in Table 6 reports the sum of the two explicit MPI synchronization costs: `mpi_mark_sync_ms + mpi_evap_sync_ms`. Their detailed values are shown in Table 7.

The results show that this MPI strategy does *not* provide a positive speedup on the evaluated platform and workload. With 2 processes, the total runtime increases from 35.93 s to 40.44 s, corresponding to a speedup of only 0.889×. With 4 processes, the runtime is almost identical to the 1-process case (0.999× speedup), which means that the reduction in ant-computation time is almost exactly canceled by communication overhead. With 8 processes, performance degrades substantially to 52.32 s, i.e., only 0.687× of the baseline speed.

Table 7. Breakdown of MPI synchronization costs for the first distributed-memory strategy.

Processes	MPI mark sync (ms)	MPI evaporation sync (ms)	Total sync (ms)
1	0.0	0.0	0.0
2	11560.6	9609.02	21169.62
4	14433.4	10416.4	24849.8
8	24573.7	17643.3	42217.0

This negative scaling is not an implementation accident; it follows directly from the structure of the first MPI method. The ant computation itself scales in the expected direction: the measured ant-update time decreases from 4046.28 ms at 1 process to 2470.86 ms, 1460.42 ms, and 1138.52 ms for 2, 4, and 8 processes, respectively. This confirms that distributing ants across processes successfully reduces local agent work. If ant traversal were the only cost, this method would indeed scale.

The problem is that the replicated-environment design replaces a large part of computation by repeated global communication. At every iteration, the full pheromone map must be synchronized twice across all processes. Since the map contains two pheromone fields over the full 513×513 grid, these collective communications move a significant amount of data even though each process only advances a fraction of the ants. The communication volume is therefore proportional to the grid size, not to the number of local ants. In other words, the algorithm reduces the *agent-side* work by partitioning ants, but keeps the *environment-side* state globally replicated and globally synchronized.

This effect is visible in the timing breakdown. At 2 processes, the explicit MPI synchronization already costs about 21.17 s in total, which is far larger than the saved ant-computation time. At 4 processes, synchronization rises further to about 24.85 s. At 8 processes, it reaches more than 42 s, which by itself already exceeds the full 1-process runtime of the whole simulation loop. Once this happens, adding more processes cannot help: the program becomes communication-dominated.

A second important observation is that the evaporation stage appears to grow dramatically with the number of processes. This does not mean that the local evaporation kernel itself became algorithmically more expensive. Rather, in the current implementation, the evaporation-related timing includes the cost of producing row-partitioned partial buffers and then merging them through a second full-grid `MPI_Allreduce(MAX)`. Therefore, the reported evaporation cost effectively reflects both local grid processing and the distributed synchronization needed to reconstruct the global evaporated pheromone map. This explains why the measured `evaporation_ms` grows from about 2.20 s with one process to 42.69 s with eight processes.

Conceptually, this experiment highlights the central trade-off of the first MPI approach. Its advantage is simplicity: load balancing across ants is easy, each process can simulate its local ants independently, and no inter-process ant migration or halo management is required. But this simplicity is obtained by replicating the full environment everywhere, which in turn forces heavy collective synchronization. Thus, the method is only attractive when the number of ants is sufficiently large, the map is sufficiently small, and the communication system is fast enough that the saved ant-computation time dominates the synchronization cost. On our machine and problem size, this condition is not satisfied.

These results also align well with the earlier shared-memory experiments. The ant phase is indeed the dominant source of useful parallel work, but it is also tightly coupled to a global pheromone state. Within a shared-memory setting, this coupling can be handled through thread-local accumulation and cheap in-memory merging. In the replicated MPI

setting, the same coupling becomes expensive because consistency must be restored through full distributed reductions. The optimization therefore moves from a *compute-bound* regime to a *communication-bound* regime.

Overall, the first distributed-memory strategy should be understood as a useful reference design rather than as the most efficient implementation. It demonstrates that evenly distributing ants is not sufficient for scalability when the shared environment remains globally replicated. The true bottleneck is no longer agent computation, but the cost of synchronizing the pheromone field at every iteration. This is precisely why the second family of distributed approaches, based on domain decomposition of the map, is algorithmically more promising: it reduces communication from full-grid global reductions to boundary exchanges, at the cost of a more complex implementation.

4.5 Effect of Sub-map Ownership and Border Exchange (Seconde Façon)

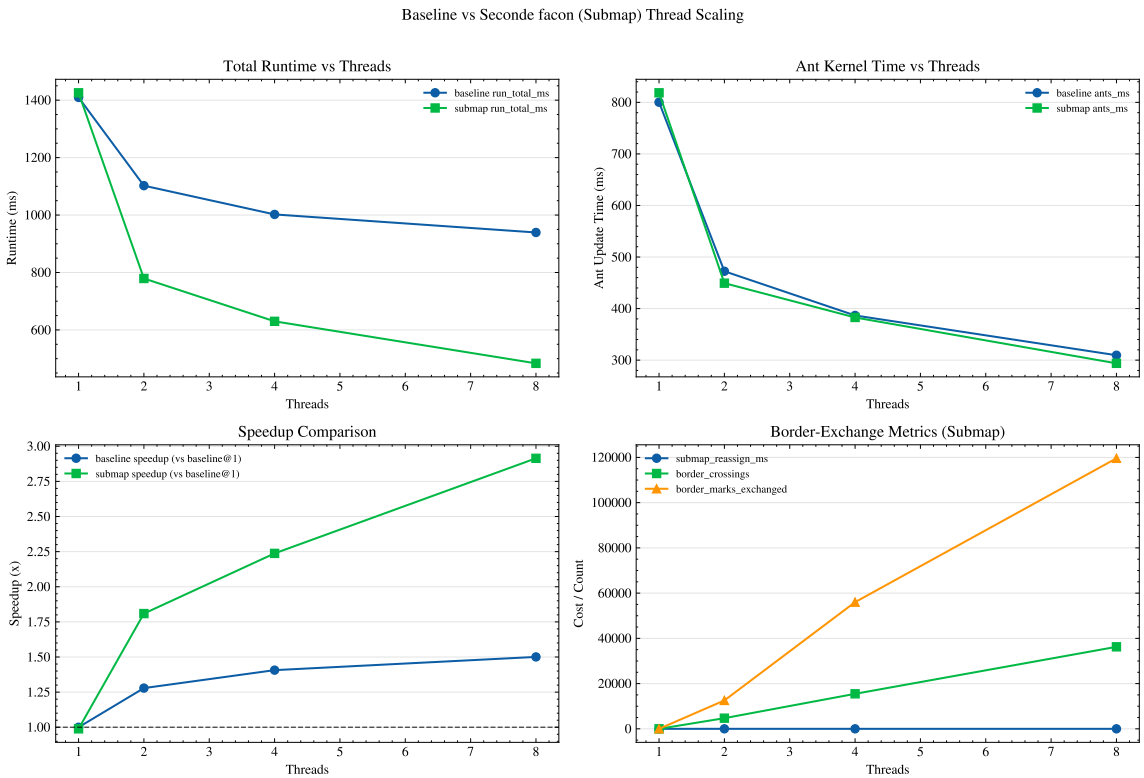

Fig. 6. **Comparison** of the vectorized+SIMD OpenMP baseline against the second-amelioration submap strategy across 1, 2, 4, and 8 threads using the same workload (‘5000‘ ants, ‘500‘ iterations, 3-run averages). The submap method is slightly slower at 1 thread due to ownership/reassignment overhead, but scales much better for multi-threaded runs, reducing total runtime from ‘1102.33‘ to ‘779.14‘ ms (2 threads), ‘1002.18‘ to ‘629.96‘ ms (4 threads), and ‘939.35‘ to ‘483.69‘ ms (8 threads). The border-exchange metrics (‘$\text{border}_crossings$‘, ‘$border_marks_exchanged$‘, and‘$submap_reassign_ms$‘)$quantifytheexplicitinter-submapcommunicationcostthatenablesthisimprovedparallelscalability.$

We now consider the second family of parallelization strategies proposed in the project statement, namely the case where each worker manages only a *part* of the map and advances only the ants currently located inside that sub-domain. In the original statement, this idea is described in a process-based distributed-memory setting. In our

implementation, however, we realize the same algorithmic principle as a *shared-memory adaptation* on top of the existing vectorized+OpenMP code: the map is partitioned into row-wise sub-maps, each sub-map is assigned to an owner (typically an OpenMP worker), and ants are dynamically assigned to the owner corresponding to their current row position.

This design preserves the central idea of the *seconde façon*: workers do not manage the full map uniformly anymore, but only the ants and updates that belong to their own sub-domain. As a consequence, communication is no longer a global synchronization over the entire pheromone map, as in the first MPI method. Instead, only boundary-related information needs to be exchanged: when an ant crosses from one sub-map to another, it will be reassigned to the new owner at the next iteration; similarly, when a pheromone mark generated by a local ant falls into another owner's region, only that mark is transferred to the target owner. Thus, the exchanged data volume is proportional to *cross-boundary activity*, rather than to the full map size.

Implementation strategy. The implemented strategy is based on five steps executed at each iteration.

First, the grid is partitioned into contiguous row blocks. An ownership function maps each ant to a sub-map according to its current row coordinate. This produces a dynamic binning of ants across owners.

Second, each owner advances only the ants currently assigned to its local sub-map. This matches the problem requirement that a worker should only manage the ants that are on its own part of the map.

Third, pheromone marks are first accumulated in local thread-private buffers. Marks targeting the same sub-map are kept locally, while marks targeting another sub-map are appended to a border-exchange buffer associated with the destination owner.

Fourth, after all local ant updates are completed, exchanged border marks are delivered to their target owners. Each owner then deduplicates and commits only the marks belonging to its own region. In this way, the algorithm avoids a full-map merge and restricts data movement to the actual boundary interactions.

Fifth, pheromone evaporation is also performed by sub-map, using a row-partitioned kernel (`do_evaporation_rows`). This removes the need for a full-grid serial evaporation pass and keeps the update local to each owner.

This strategy has two conceptual advantages. On the one hand, it follows much more closely the intended *domain ownership* model of the seconde façon than the previous replicated-map approach. On the other hand, it avoids the large global reductions that made the first MPI strategy communication-bound. Its main weakness is potential load imbalance: if ants concentrate around the nest or around a limited region of the map, the corresponding owner may receive much more work than others. However, when the ant population is sufficiently large and the exploration coefficient is not too small, ants tend to spread across the environment, making the distribution more balanced in practice.

Table 8 shows that the sub-map ownership strategy introduces a small overhead in the purely sequential case, but becomes increasingly beneficial as the number of threads grows. With one thread, the sub-map version is slightly slower than the baseline (1425.58 ms vs. 1409.58 ms). This is expected: when no actual parallel ownership exists, the additional work of ant reassignment, owner binning, and border-management logic becomes pure overhead. Indeed, the measured reassignment cost is about 4.69 ms, while no border crossings or exchanged marks occur in a meaningful parallel sense.

Once multiple threads are used, however, the picture changes substantially. At 2 threads, the total runtime decreases from 1102.33 ms for the baseline to 779.14 ms for the sub-map strategy, corresponding to a speedup of about 1.41× over the baseline shared-memory version. At 4 threads, runtime further drops from 1002.18 ms to 629.96 ms, i.e., about 1.59× faster. At 8 threads, the advantage becomes even larger: 483.69 ms versus 939.35 ms, corresponding to nearly 1.94× speedup over the baseline.

Table 8. Comparison between the baseline shared-memory version and the sub-map ownership strategy (Seconde façon analogue). Experiments use 5000 ants, 500 iterations, and 3 runs; reported values are averages.

Threads	Mode	Run total (ms)	Ants (ms)	Select (ms)	Mark (ms)	Reassign (ms)	Border crossings	Border marks
1	Baseline	1409.58	800.255	285.584	547.544	0.000	0	0
1	Sub-map	1425.58	818.505	293.429	543.938	4.685	0	0
2	Baseline	1102.33	472.402	327.093	565.141	0.000	0	0
2	Sub-map	779.143	449.315	289.191	284.354	4.935	4713	12555
4	Baseline	1002.18	386.671	365.877	553.512	0.000	0	0
4	Sub-map	629.959	382.762	388.142	192.622	2.628	15477	56007
8	Baseline	939.345	309.509	609.763	565.485	0.000	0	0
8	Sub-map	483.690	293.792	547.353	134.776	2.613	36258	119601

A key observation is that the ant-update time itself decreases only moderately, from 309.51 ms to 293.79 ms at 8 threads. The dominant improvement instead comes from the pheromone-marking related cost. For example, the measured marking time drops from 565.49 ms in the baseline to only 134.78 ms in the 8-thread sub-map version. This is a very important result: the benefit of the seconde façon is not primarily that each ant suddenly becomes cheaper to update, but rather that the ownership-based organization greatly reduces contention and unnecessary global sharing during pheromone writes. In the baseline parallel implementation, thread-local accumulation must ultimately be merged into a more globally shared structure. In the sub-map design, each owner is responsible only for its own region, so most marks remain local and only the truly cross-boundary marks need to be exchanged.

This interpretation is supported by the new trace metrics. The number of border crossings and exchanged border marks increases with the number of threads: from 4713 crossings and 12555 exchanged marks at 2 threads to 36258 crossings and 119601 exchanged marks at 8 threads. This confirms that the implementation is indeed capturing the intended boundary-exchange behavior explicitly. At the same time, the reassignment overhead remains very small (around 2.6–4.9 ms), which means that the bookkeeping cost of ownership migration is negligible compared with the savings obtained from more localized updates.

From an algorithmic point of view, these results match the original motivation of the seconde façon. Compared with the first MPI method, which replicated the whole environment and paid for repeated global synchronization, the sub-map ownership strategy localizes both state and communication. Compared with the baseline shared-memory implementation, it reduces the amount of effectively shared write traffic by turning the pheromone field into a mostly owner-local structure. The price to pay is potential load imbalance, since ants are not guaranteed to distribute uniformly across sub-maps. This effect is not dominant in our experiments, presumably because the number of ants is large enough for the workload to spread reasonably well over the map.

Overall, the seconde façon appears to be a more scalable structural optimization than the first distributed-memory strategy, at least in its shared-memory analogue implemented here. Its strength is not simply “more threads”, but a better *ownership model*: computation, writes, and communication follow the spatial locality of the simulated world. This is a deeper improvement than raw loop parallelization, because it changes the organization of the algorithm to better reflect the locality of both ants and pheromone updates.

5 Conclusion

In this report, we studied the implementation and parallelization of an ant-colony-based foraging simulator evolving on a fractal terrain. The model combines two types of computation: a large number of agent-level updates, corresponding to ant movement and local decision making, and repeated grid-level updates, corresponding mainly to pheromone evaporation and propagation. This structure makes the simulator a natural case study for evaluating several forms of parallelism.

Our first set of measurements showed that the ant-update phase is by far the dominant cost of the workflow, while pheromone evaporation is the main secondary cost and the remaining stages are comparatively negligible. This observation already provides the central performance lesson of the project: the main optimization target is not initialization or visualization, but the repeated per-ant computation performed at each iteration.

The SIMD vectorization experiments demonstrated that regular grid-based kernels benefit the most from instruction-level parallelism. In particular, pheromone evaporation achieved a clear speedup thanks to its dense and branch-light structure. The ant-update phase also improved, but more modestly, because its control flow, random decisions, and irregular memory accesses limit the effectiveness of vectorization. Therefore, SIMD should be understood as an important but partial optimization: it improves efficiency inside regular kernels, but it does not remove the main source of sequential work.

The shared-memory OpenMP implementation provided the strongest performance improvement on the baseline simulator. By parallelizing ant traversal across threads and preserving correctness through thread-local accumulation and merging, it significantly reduced the cost of the dominant computation phase. This confirms that the most profitable source of parallelism in the simulator lies across ants rather than inside the logic of a single ant.

We then evaluated two more structural parallelization strategies. The first one, based on a replicated environment and MPI reduction, is simple and well balanced from the point of view of ant distribution, but it scales poorly because consistency of the pheromone fields must be restored through repeated full-map collective synchronization. On the tested platform, the communication overhead dominates the saved ant computation and prevents positive speedup. The second one, implemented here as a shared-memory analogue of the sub-map ownership strategy, follows a different principle: each owner advances only the ants currently in its own region, and only cross-owner activity is exchanged. This design better matches the spatial locality of the simulated environment. Although it adds some management overhead in the single-thread case, it yields substantially better performance in the multi-threaded setting by reducing contention and localizing pheromone writes.

Taken together, the results suggest three main conclusions. First, optimization efforts should focus primarily on the ant-update phase, since this is both the dominant cost and the component whose complexity grows with the number of ants. Second, SIMD and thread-level parallelism are complementary: SIMD improves regular kernels, while OpenMP removes the main sequential bottleneck by exploiting agent-level parallelism. Third, locality-aware decompositions of the environment are more promising than globally replicated state when moving toward more scalable parallel designs.

Several extensions are possible in future work. A full distributed-memory implementation of the second strategy would make it possible to compare the shared-memory analogue presented here with a true process-based sub-domain decomposition. Further improvements could also include better load balancing between owners, more efficient handling of cross-boundary marks, and a deeper analysis of scalability as a function of both the number of ants and the map size. Nevertheless, the present study already shows clearly that the performance of parallel ACO-style simulators depends

not only on adding more threads or processes, but on choosing a decomposition strategy that respects the locality and synchronization structure of the underlying algorithm.

6 Misc.

The project structure has been reorganized to improve clarity and reproducibility. In particular, all generated outputs are now stored in the `output/` directory. In addition, instead of using a standard thesis template, the document structure has been adjusted to better match the specific requirements and scope of this project.

The source code of this project can be found at https://github.com/Languisher/CSC_4OS02_TA-Systemes-Paralleles-et-Distribues.